

Homework 1 — From Derivatives to Gradient Descent

CSCI1409 Deep Learning — Week 1 — Aidyn Abirov

- **Tasks:** 1–6
 - **Deadline:** Before the next lecture
 - **Submission:** Send your completed homework **directly to me on Microsoft Teams**.
-

Important Rules

1. AI is not allowed

You must complete this homework **without using AI tools**.

Do **not** use:

- ChatGPT
- GitHub Copilot
- Claude
- Gemini
- Microsoft Copilot
- AI-generated solutions or code
- Other AI assistants

However, **using Google and other search engines is allowed**. You may search for documentation, explanations, examples, and references.

Important: Do not use **Google AI Overviews** or other AI-generated search summaries. If Google displays an AI-generated answer at the top of the search results, **do not use it**.

The goal is to make sure that you personally understand the concepts and can implement them yourself.

2. Homework check during practice

Your homework will be **checked during the practice session**. This will be a short individual check where you may be asked to:

- explain your solution;
- explain specific lines of code;
- modify a small part of your code;
- correct a mistake;
- reproduce part of your solution;
- answer conceptual questions about your implementation.

You should therefore be able to **explain everything you submit**. The check will take place **during the practice session**, not necessarily at the beginning.

3. Strict deadline

Your homework must be submitted **before the next lecture begins**.

Once the next lecture starts, no homework submissions will be accepted.

Please submit your work early enough to avoid technical problems at the deadline.

Submission Requirements

Submit your completed homework **directly to me on Microsoft Teams**.

For each task:

- Complete the provided code.
- Run all cells before submitting.
- Include the requested plots and numerical results.
- Answer conceptual questions in **Markdown cells**.
- Keep your code readable.
- Use meaningful variable names.
- Do not use automatic differentiation libraries such as PyTorch, TensorFlow, or JAX for these tasks.

You may use:

- Python
- NumPy
- Matplotlib
- Google/search engines and official documentation

AI-generated solutions are not allowed.

Task 1 — The Numeric Derivative

Concept

A derivative measures the local rate of change of a function. You will see that a smaller step size h is **not always better** when computing a derivative numerically.

Problem

Implement forward and central finite differences:

$$\text{Forward: } \frac{f(x+h) - f(x)}{h}$$

$$\text{Central: } \frac{f(x+h) - f(x-h)}{2h}$$

Test both methods on $f(x) = \sin(x)$

at $x = 1$.

The true derivative is:

$$f'(1) = \cos(1) \approx 0.540302$$

Then:

1. Sweep h from 10^{-1}
2. to 10^{-12} .
3. Calculate the absolute error for both methods.
4. Plot the error against h using **log-log axes**.
5. Find the value of h that gives the smallest error for each method.
6. Explain why making h smaller eventually makes the numerical derivative worse.

Starter Code

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.sin(x)

def df_true(x):
    return np.cos(x)

def deriv_forward(f, x, h):
    # TODO
    ...

def deriv_central(f, x, h):
    # TODO
    ...

x0 = 1.0

print("true      :", df_true(x0))
```

```

print("forward h=1e-5:", deriv_forward(f, x0, 1e-5))
print("central h=1e-5:", deriv_central(f, x0, 1e-5))

hs = np.logspace(-1, -12, 45)

err_fwd = np.array([
    abs(deriv_forward(f, x0, h) - df_true(x0))
    for h in hs
])

# TODO: calculate central-difference errors
err_ctr = ...

plt.figure(figsize=(6, 4))
plt.loglog(hs, err_fwd, "o-", label="forward")
plt.loglog(hs, err_ctr, "s-", label="central")
plt.gca().invert_xaxis()

plt.xlabel("h")
plt.ylabel("absolute error")
plt.legend()
plt.title("Finite-Difference Error vs Step Size")
plt.tight_layout()
plt.show()

print("best h, forward:", hs[np.argmin(err_fwd)])
print("best h, central:", hs[np.argmin(err_ctr)])

assert abs(
    deriv_central(f, x0, 1e-5) - df_true(x0)
) < 1e-7

print("Task 1 OK")

```

Questions

Answer the following in your notebook:

1. Which method gives a smaller error near its optimal h ?
2. Why does the error eventually increase when h becomes extremely small?
3. Why is the central-difference method generally more accurate than the forward-difference method?

Stretch Goal (optional)

Repeat the experiment using $f(x) = x^2$,

$x = 3$.

Explain why the central-difference curve behaves differently from the $\sin(x)$ experiment.

Then try $f(x) = |x|$, $x = 0$. Report the forward and central estimates and explain what happens at the corner.

Task 2 — Gradient of a Scalar Function

Concept

The derivative of a function with multiple inputs is a **gradient** — a vector containing the partial derivative with respect to every input. The gradient points in the direction of **steepest ascent**.

Problem

Consider:

$$f(w_1, w_2) = w_1^2 + 3w_1w_2 + 2w_2^2$$

You must:

1. Differentiate the function **by hand**.
2. Write the analytical gradient in your notebook.
3. Implement `numeric_gradient()`.
4. Compare the analytical and numerical gradients at $w = [1, 2]$.
5. Experimentally verify that the gradient points in the direction of steepest ascent.

Starter Code

```
import numpy as np

rng = np.random.default_rng(0)

def f(w):
    return w[0]**2 + 3*w[0]*w[1] + 2*w[1]**2

def grad_analytic(w):
    # TODO: calculate the two partial derivatives
    return np.array([..., ...])

def numeric_gradient(f, w, h=1e-5):
    g = np.zeros_like(w, dtype=float)

    for i in range(len(w)):
```

```

    w_plus = w.copy()
    w_minus = w.copy()

    # TODO:
    # perturb ONLY coordinate i
    # calculate the central difference

    ...

    return g

w = np.array([1.0, 2.0])

ga = grad_analytic(w)
gn = numeric_gradient(f, w)

print("f(w)          =", f(w))
print("analytic grad =", ga)
print("numeric grad  =", gn)

print(
    "max abs diff =",
    np.max(np.abs(ga - gn))
)

eps = 1e-3

d_grad = ga / np.linalg.norm(ga)

gain_grad = f(w + eps * d_grad) - f(w)

gains_random = []

for _ in range(20):

    d = rng.normal(size=2)
    d /= np.linalg.norm(d)

    gains_random.append(
        f(w + eps * d) - f(w)
    )

print("gain along gradient :", gain_grad)
print("best random gain    :", max(gains_random))

```

```
assert np.allclose(ga, [8.0, 11.0])
assert np.max(np.abs(ga - gn)) < 1e-6

assert gain_grad >= max(gains_random) - 1e-9

print("Task 2 OK")
```

Questions

Explain:

1. What is the difference between a derivative and a gradient?
2. Why do we use `.copy()` when creating `w_plus` and `w_minus`?
3. Why must we perturb one coordinate at a time when calculating the gradient?
4. What does it mean that the gradient points in the direction of steepest ascent?

Stretch Goal (optional)

Create a contour plot of $f(w_1, w_2)$ for $w_1, w_2 \in [-3, 3]$. Use `plt.contour()` and overlay the gradient at `[1, 2]` using `plt.quiver()`.

Explain why the gradient is perpendicular to the contour line.

Task 3 — The Chain Rule: By Hand and in Code

Concept

The chain rule allows us to differentiate a composition of functions. This is the basic idea behind **backpropagation**.

Consider the neuron:

$$z = wx + b$$

$$a = \sigma(z)$$

$$L = (a - y)^2$$

Problem

Using:

```
x = 2.0
y = 1.0
w = 0.3
b = -0.1
```

you must:

1. Derive the gradient **by hand**.
2. Write down $\partial L / \partial a$, $\partial a / \partial z$, $\partial z / \partial w$, and $\partial z / \partial b$.
3. Multiply the derivatives using the chain rule.
4. Implement the forward pass.
5. Implement the analytical gradient.
6. Verify the result using a numerical gradient check.

Starter Code

```
import numpy as np
```

```
def sigmoid(z):  
    return 1.0 / (1.0 + np.exp(-z))
```

```
def forward(w, b, x, y):
```

```
    z = ...          # TODO  
    a = ...          # TODO  
    L = ...          # TODO
```

```
    return z, a, L
```

```
def grad_analytic(w, b, x, y):
```

```
    z, a, L = forward(w, b, x, y)
```

```
    dL_da = ...      # TODO  
    da_dz = ...      # TODO  
    dz_dw = ...      # TODO  
    dz_db = ...      # TODO
```

```
    dL_dw = dL_da * da_dz * dz_dw  
    dL_db = dL_da * da_dz * dz_db
```

```
    return dL_dw, dL_db
```

```
def numeric_gradient_wb(w, b, x, y, h=1e-5):
```

```
    # TODO:  
    # use central differences to estimate dL/dw and dL/db  
    # (perturb w and b one at a time, holding the other fixed)  
    ...
```

```
x, y = 2.0, 1.0
```

```

w, b = 0.3, -0.1

z, a, L = forward(w, b, x, y)
print("z, a, L =", z, a, L)

dL_dw, dL_db = grad_analytic(w, b, x, y)
print("analytic dL/dw, dL/db =", dL_dw, dL_db)

ndw, ndb = numeric_gradient_wb(w, b, x, y)
print("numeric dL/dw, dL/db =", ndw, ndb)

assert abs(dL_dw - ndw) < 1e-6
assert abs(dL_db - ndb) < 1e-6

print("Task 3 OK")

```

Questions

Explain:

1. What is the chain rule doing in this example?
2. Why does the gradient contain several factors multiplied together?
3. Why is $\partial L / \partial w$ exactly twice $\partial L / \partial b$ for these particular values?
4. Why is numerical gradient checking useful?

Stretch Goal (optional)

Replace the sigmoid with ReLU. Run the experiment with $b = -0.7$ and then with $b = -0.6$. Explain what happens when $z = 0$.

Task 4 — Forward Pass of a Tiny Linear Model

Concept

A machine-learning model processes multiple examples at once. For a linear model:

$$\hat{y} = Xw + b$$

Correct **array shapes** are essential.

Problem

Implement `predict(X, w, b)`. Then deliberately create a shape bug and investigate what happens.

Starter Code

```
import numpy as np

# X: 4 examples, 2 features each
X = np.array([
    [1.0, 2.0],
    [2.0, 0.5],
    [3.0, 1.0],
    [4.0, 3.0],
])

w = np.array([2.0, -1.0]) # shape (2,)
b = 0.5

def predict(X, w, b):
    # TODO: return X @ w + b, shape should be (4,)
    ...

yhat = predict(X, w, b)
print("yhat shape:", yhat.shape)
print("yhat      :", yhat)

# --- Now deliberately introduce a shape bug ---
w_bad = w.reshape(2, 1) # shape (2, 1) instead of (2,)

yhat_bad = X @ w_bad + b
print("yhat_bad shape:", yhat_bad.shape)
print("yhat_bad      :", yhat_bad)

assert yhat.shape == (4,)
assert yhat_bad.shape == (4, 1)

print("Task 4 OK")
```

Questions

Explain:

1. What does each dimension of X represent?
2. What does the shape $(2,)$ of w represent?
3. Why is the correct prediction shape $(4,)$?
4. Why does changing w to shape $(2,1)$ change the prediction shape?
5. Why can a shape such as $(4,1)$ be dangerous even though the values look correct?

Stretch Goal (optional)

Generalise the model to multiple outputs. Let $W \in \mathbb{R}^{m \times 2}$, $b \in \mathbb{R}^m$. Implement the forward pass and explain the resulting shape of $\hat{y}$.

Task 5 — Computing the MSE Loss

Concept

The loss function converts the performance of a model on many examples into **one scalar number**. For Mean Squared Error:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Problem

Using the predictions from Task 4:

1. Implement MSE using a Python loop.
2. Implement MSE using NumPy vectorisation.
3. Verify that both implementations produce the same result.
4. Calculate MSE using the incorrectly shaped $(4, 1)$ predictions.
5. Explain why the resulting value is wrong.

Questions

1. Calculate the residuals by hand.
2. Calculate the MSE by hand.
3. Why do the loop and vectorised implementations give the same answer?
4. Why does subtracting $(4,)$ from $(4, 1)$ produce an array of shape $(4, 4)$?
5. Why is this a dangerous bug?

Stretch Goal (optional)

Add a shape check:

```
assert yhat.shape == y.shape
```

Then implement Mean Absolute Error:

$$\text{MAE} = \frac{1}{N} \sum_i |\hat{y}_i - y_i|$$

Calculate the MAE for the same predictions. Finally, change the last target from -2 to -20 . Which loss changes more dramatically — MSE or MAE? Explain why.

Task 6 — One Gradient-Descent Step

Concept

You are now going to train a model. The complete learning process is:

Forward Pass → Loss → Gradient → Update

This is the fundamental loop behind neural-network training.

Problem

Use the dataset $\{(1,2), (2,4), (3,5)\}$ with the model $\hat{y} = wx + b$ and MSE loss.

Start with:

$w = 0$

$b = 0$

learning rate = 0.1

You must:

1. Calculate the initial loss.
2. Calculate $\partial L / \partial w$ and $\partial L / \partial b$ analytically.
3. Verify both gradients numerically.
4. Perform exactly **one** gradient-descent step.
5. Calculate the new loss.
6. Verify that the loss decreased.

The analytical gradients are:

$$\frac{\partial L}{\partial w} = \frac{2}{N} \sum_i (\hat{y}_i - y_i) x_i$$

$$\frac{\partial L}{\partial b} = \frac{2}{N} \sum_i (\hat{y}_i - y_i)$$

Questions

Answer the following:

1. Why are both gradients negative?
2. Why does subtracting a negative gradient increase w and b ?
3. Why must both gradients be calculated **before** updating the parameters?
4. What happens if you use $w + \eta \cdot gw$ instead of $w - \eta \cdot gw$?
5. Why is checking whether the loss decreased a useful debugging tool?

Stretch Goal (optional)

Train the model for 200 iterations and plot the loss over time. Then experiment with $\eta = 1.0$ and compare it with $\eta = 0.1$. Explain why a learning rate that is too large can make training unstable.

Final Reflection

After completing Tasks 1–6, answer these questions in **5–8 sentences**:

1. What is the relationship between a derivative and a gradient?
 2. What does the chain rule have to do with backpropagation?
 3. Why is numerical gradient checking useful?
 4. Why are array shapes important in machine learning?
 5. What are the four main steps of gradient descent?
-

Final Checklist

Before submitting, make sure:

- ☐ Tasks 1–6 are completed.
- ☐ All code runs without errors.
- ☐ All required plots are included.
- ☐ All questions are answered.
- ☐ You have run the notebook from beginning to end.
- ☐ You understand and can explain your submitted work.
- ☐ You have **not used AI** to complete the homework.
- ☐ If you used Google, you did **not** use Google AI Overview or other AI-generated summaries.
- ☐ You sent the homework **directly to me on Teams**.
- ☐ You submitted it **before the next lecture began**.

After the next lecture starts, no homework submissions will be accepted.

Remember: Your homework will be checked **during the practice session**, so make sure you understand your own solution.